

Reproducing the `my_surface_iter` GP surface-bundle scaling loop

This note describes the **iterative GP surface scaling** workflow that produced the archived workspace `runs/my_surface_iter/`: alternating a full `run_gp` fit with spectroscopic **per-spec_bundle_id** linear rescaling against the GP latent surface on `X_fill` (default `mu_raw`).

Prerequisites

- **Python** 3.10+ recommended
- **Packages:** `numpy`, `scipy`, `matplotlib`, `george`
- Working directory: **repository root** (`gp/`), so imports and default `runs/` paths resolve correctly.

```
cd /path/to/gp
pip install numpy scipy matplotlib george
```

Inputs used for `my_surface_iter`

Role	File (paths relative to repo root)
Starting training bundle	<code>gp_work_scaled_nophot_m8767_m8217.npz</code> (or whatever you pass as <code>--input-bundle</code>)
Grid / flux normalization meta	<code>gp_scaled_bundle_meta.json</code> (<code>grid_norm_info</code> required)

The driver copies the input bundle to `runs/my_surface_iter/iter_00/bundle.npz`.

What the driver does (high level)

1. For each outer iteration `k = 0 ... K-1`:
 - Runs `run_gp.py` on `runs/my_surface_iter/iter_{k:02d}/bundle.npz` with tag `surf_iter_k{k:02d}` under `runs/` (or your `--runs-dir`).
 - Copies `predictions.npz` and `config.json` into the workspace iteration folder.
 - Interpolates `mu_raw` (or `mu`) from `X_fill` onto spectroscopic training rows, solves a **closed-form WLS** linear scale per `spec_bundle_id`, clips multipliers, and applies them via `bundle_scale_pipeline.apply_epoch_linear_multiplier` (photometry unchanged). Writes `iter_{k+1}/bundle.npz`.
2. For `k >= 1`, `run_gp` is warm-started from `runs/<gp-tag-prefix>_k{k-1:02d}/config.json` when present (`--warm-start-config-json`).
3. Appends one JSON line per iteration to `iteration_log.jsonl`; after the loop, writes `metrics/*.png`, `scaling/` CSVs and evolution plots for highlight bundles (default 3 and 5 if `--diag-bundles` omitted).

Shell variables (recommended)

Set these once from the **repository root** (after `cd`), then reuse them in every command below. Adjust paths if your layout differs.

```
cd /path/to/gp
GP_ROOT="$(pwd)"

BUNDLE="${GP_ROOT}/gp_work_scaled_nophot_m8767_m8217.npz"
META="${GP_ROOT}/gp_scaled_bundle_meta.json"

WORKSPACE="${GP_ROOT}/runs/my_surface_iter"
RUNS_DIR="${GP_ROOT}/runs"
GP_TAG_PREFIX="surf_iter"

# Last outer iteration (0-based; 19 matches iter_19/ and surf_iter_k19)
ITER_LAST="${ITER_LAST:-19}"
printf -v GP_TAG '%s_k%02d' "${GP_TAG_PREFIX}" "${ITER_LAST}"
WORK_BUNDLE="${WORKSPACE}/iter_${ITER_LAST}/bundle.npz"

# plot_results / compare_runs: parent of <tag>/ (same as run_gp --output-dir)
export GP_RUNS_DIR="${RUNS_DIR}"
```

Optional: `export GP_RUNS_DIR="${RUNS_DIR}"` so helpers that honor it (for example `gp_scales.py`) resolve the runs directory without extra flags. `plot_results.py` still needs `--output-dir "${RUNS_DIR}"` as in the examples below.

Command that matches the archived `my_surface_iter` layout

Run this **after** the **Shell variables** block so `${BUNDLE}`, `${GP_ROOT}`, etc. are defined.

Tags under `runs/` are `surf_iter_k00 ...` (prefix `surf_iter`). Workspace directory `my_surface_iter`. Adjust paths if your clone lives elsewhere.

Important: Flags `--diag-bundles`, `--diag-full-overview-interval`, `--plot-results-each-iter` belong to `iterate_gp_surface_bundle_scale.py` and must appear **before** a lone `--`. Only `run_gp.py` options go **after** `--`.

```
cd "${GP_ROOT}"

python3 iterate_gp_surface_bundle_scale.py \
  --input-bundle "${BUNDLE}" \
  --meta "${META}" \
  --workspace "${WORKSPACE}" \
  --runs-dir "${RUNS_DIR}" \
  --gp-tag-prefix "${GP_TAG_PREFIX}" \
  --max-iters 20 \
  --run-gp-max-iter 60 \
  --surface-mu-key mu_raw \
  --diag-bundles 3,5 \
  --diag-full-overview-interval 5 \
```

```

--plot-results-each-iter \
--converge-max-log-scale 5e-4 \
--converge-delta-chi2-spec 5e-4 \
--bundle-scale-clip 10 \
-- \
--additive-time --additive-wls \
--kernel-time matern52 --kernel-wls matern52 \
--mean linear --optimize \
--meta-json "${META}"

```

Lighter run (smoke test)

Uses **BUNDLE**, **META**, and **RUNS_DIR** from the shell-variables block above.

```
cd "${GP_ROOT}"
```

```
SMOKE_WORKSPACE="${GP_ROOT}/runs/smoke_surface_iter"
SMOKE_PREFIX="smoke_surf"
```

```

python3 iterate_gp_surface_bundle_scale.py \
-i "${BUNDLE}" \
--meta "${META}" \
-w "${SMOKE_WORKSPACE}" \
--runs-dir "${RUNS_DIR}" \
--gp-tag-prefix "${SMOKE_PREFIX}" \
--max-iters 1 \
--run-gp-max-iter 10 \
--diag-bundles 3,5 \
-- \
--additive-time --additive-wls \
--kernel-time matern52 --kernel-wls matern52 \
--mean linear --optimize \
--meta-json "${META}"

```

Where outputs land

Location	Contents
<code>\${WORKSPACE}/iter_XX/bundle.npz</code>	Working bundle per outer iteration
<code>\${WORKSPACE}/iter_XX/predictions.npz</code>	Copy of GP predictions for that iteration
<code>\${WORKSPACE}/iter_XX/config.json</code>	Copy of <code>run_gp</code> config + metrics
<code>\${WORKSPACE}/iteration_log.jsonl</code>	One JSON object per iteration (scales, chi-squared, kernel dict, etc.)
<code>\${WORKSPACE}/metrics/</code>	Summary plots after the outer loop finishes (<code>chi2_vs_iter.png</code> , ...)
<code>\${WORKSPACE}/scaling/</code>	Per-epoch cumulative scale CSVs + PNGs for highlight bundles
<code>\${WORKSPACE}/iter_XX/figs/overview/</code>	Targeted <code>plot_bands_gp_overview</code> when <code>--diag-bundles</code> is set

Location	Contents
<code>\${WORKSPACE}/iter_XX/figs/overview_full/</code>	Full overview when <code>--diag-full-overview-interval > 0</code>
<code>\${RUNS_DIR}/surf_iter_kXX/</code>	Each full <code>run_gp</code> run (predictions, config, figs/ if <code>plot_results</code> was invoked)

Post-processing: scales, standard plots, overview

Use the **same** `--runs-dir` / `--output-dir` as `run_gp` if you did not use the repo default (see `plot_results.py` help). `GP_TAG` must match `run_gp -t` (here: `${GP_TAG_PREFIX}_k${ITER_LAST}` with zero padding).

```
cd "${GP_ROOT}"

# Kernel / length-scale summary from a finished surf_iter tag
python3 gp_scales.py "${GP_TAG}"

# Standard diagnostics for one outer iteration's GP tag
python3 plot_results.py \
  --tag "${GP_TAG}" \
  --output-dir "${RUNS_DIR}" \
  --bundle "${WORK_BUNDLE}" \
  --meta "${META}" \
  --heatmap-raw

# Band + bundle overview (train posterior)
python3 plot_bands_gp_overview.py \
  --bundle "${WORK_BUNDLE}" \
  --meta "${META}" \
  --tag "${GP_TAG}" \
  --runs-dir "${RUNS_DIR}" \
  --expect-pipeline-bundle

# Optional: raw fill-grid mu panel
GRIDRAW_OUT="${WORKSPACE}/iter_${ITER_LAST}/figs/overview_grid_raw"
python3 plot_bands_gp_overview.py \
  --bundle "${WORK_BUNDLE}" \
  --meta "${META}" \
  --tag "${GP_TAG}" \
  --runs-dir "${RUNS_DIR}" \
  --expect-pipeline-bundle \
  --posterior-kind grid_raw \
  --output-dir "${GRIDRAW_OUT}"
```

Notes on bundle 3 and fit quality

Residual structure in `spec_bundle_3_*` panels can reflect **intrinsic data / cross-arm scaling limits** as much as kernel hyperparameters. If chi-squared and bundle multipliers stabilize but one

bundle still disagrees with the GP slice, treat that as a **data / pipeline** hypothesis until a narrower wavelength kernel or different preprocessing is justified.

Reproducibility archive

The zip `my_surface_iter_repro_bundle.zip` (produced next to this doc in the repo) contains:

- `runs/my_surface_iter/` — full workspace (all `iter_*`, logs, metrics, scaling, figs)
- `gp_scaled_bundle_meta.json` — meta used with the scaled bundle
- This markdown and its PDF copy

Re-running the **exact** outer loop from scratch still requires the **rest of this repository** (`iterate_gp_surface_bundle_scale.py`, `run_gp.py`, `bundle_scale_pipeline.py`, `gp_utils.py`, ...) and the **original** scaled NPZ if you do not rely on `iter_00/bundle.npz` inside the workspace.

Optional: to archive raw `run_gp` directories as well (not required to inspect the saved workspace), add `runs/surf_iter_k*` to a separate zip (~tens of MB per 20 iterations).